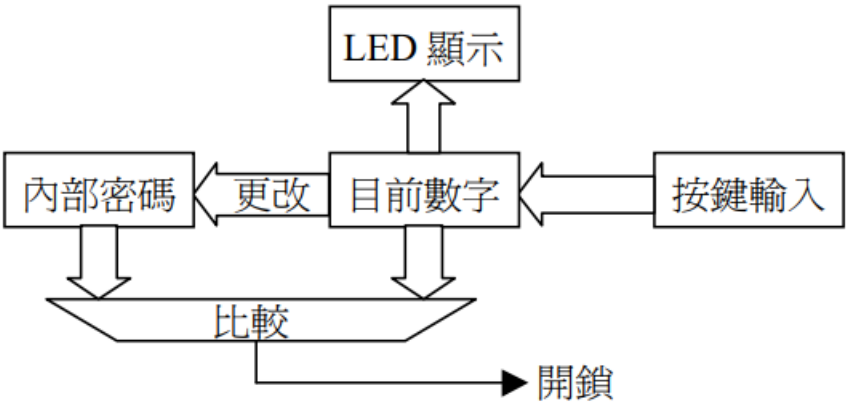
數位邏輯設計 作業十一

第十二章 密碼鎖專題

通訊 4A 0106c033 桑瑋佑

一 實驗目的:

本章實驗的目標，是透過按鍵矩陣和 LED 矩陣的控制，逐步建立一個數字式的電子密碼鎖，除了 0~9 數字以外，還新增功能鍵，分別為按鍵退位、按鍵清除、密碼清除、密碼更改、鎖位復歸和密碼檢查。其密碼鎖的系統方塊圖如下:



二 步驟:

實驗時，先用 LoadPGA 程式將 ch11.rbt 的電路檔載入 FPGA 中，依定義來控制按鍵，分別實現數字和功能鍵。

首先輸入原始密碼 0000 來解鎖，解鎖後將密碼更改成 4321，先輸入 4321 後按下密碼更改，接著按下按鍵清除後再按下鎖位復歸，輸入更改後密碼 4321，正確解鎖後會閃爍。

三 結果:以下為程式碼，為 CH12.vhd 檔

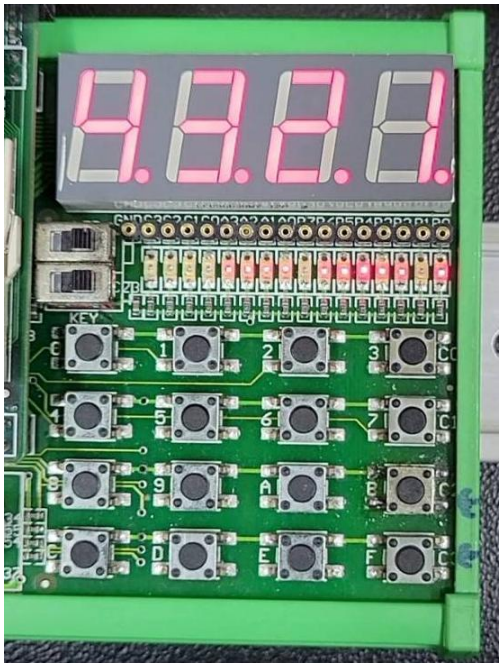
```
--The IEEE standard 1164 package, declares std_logic, rising_edge(), etc.
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_arith.all;
use IEEE.std_logic_unsigned.all;

--
--*********************************************************
-- KEYBOARD : one-bit keyboard debouncing
--                                designed by Pei-Chong Tang, Feb. 1999
--*********************************************************
--
entity KEYBOARD is
        port
        (       CLK       : in    STD_LOGIC;                                --system clock
                SAMPLE : in    STD_LOGIC;                                 --sample point
                SEQ       : in    STD_LOGIC_VECTOR(1 downto 0); --scanning sequence
                INP       : in    STD_LOGIC_VECTOR(3 downto 0); --keyboard input
                CODE      : out STD_LOGIC_VECTOR(3 downto 0); --keyboard output
                KIN       : out STD_LOGIC                                --keyboard strike
        );
end KEYBOARD;

architecture KEYBOARD_ARCH of KEYBOARD is
        signal KBUF : STD_LOGIC_VECTOR (4 downto 0);
        signal PLS1,PLS2,PLS3 : STD_LOGIC;
        signal KEY,FLT              : STD_LOGIC;
begin
--
--*********************************************************
--
-- keyboard sequence control
--
KEYBOARD_SEQUENCE : block
begin
        process (CLK)
```

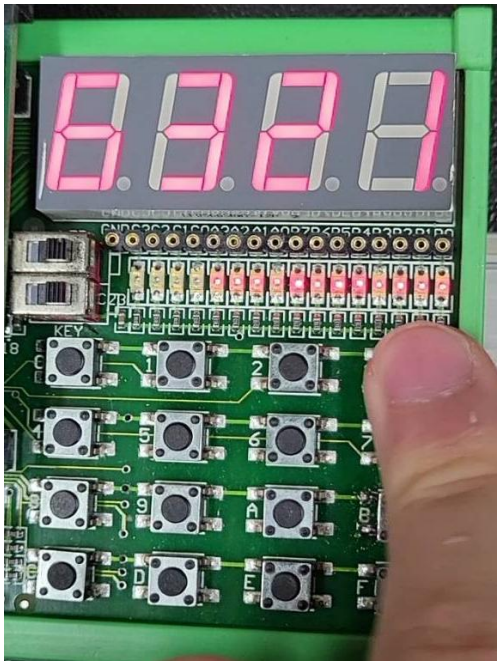
```
        begin
                if CLK'event and CLK='1' then                           --CLK rising
                        PLS3<=PLS2; PLS2<=PLS1; PLS1<=SAMPLE;       --delay
                end if;
        end process;
end block KEYBOARD_SEQUENCE;
--
--*******************************************************
--
-- keyboard decoding
--
KEYBOARD_DECODING : block
        signal D            : STD_LOGIC_VECTOR (2 downto 0);
        signal EC,RST : STD_LOGIC;
begin
        process (CLK,RST)
        begin
                if RST='1' then
                        KBUF <= "00000";
                elsif CLK'event and CLK='1' then
                        if EC='1' then
                                KBUF <= D(2) &    not D(1 downto 0) & not SEQ;
                        end if;
                end if;
        end process;
        D(2 downto 0) <= "100" when INP="0001" else
                                "101" when INP="0010" else
                                "110" when INP="0100" else
                                "111" when INP="1000" else
                                "000";
        EC    <= PLS1 and D(2);
        RST <= PLS3 and not SEQ(1) and not SEQ(0);
end block KEYBOARD_DECODING;
--
--*******************************************************
--
-- keyboard latch
--
```

```
KEYBOARD_LATCH : block
        signal EC : STD_LOGIC;
begin
        process (CLK)
        begin
                if CLK'event and CLK='1' then               --CLK rising
                        if EC='1' then                              --CLK enable
                                CODE <= KBUF(3 downto 0);    --latch code
                                KEY    <= KBUF(4);
                        end if;
                end if;
        end process;
        EC <= PLS2 and not SEQ(1) and not SEQ(0);
end block KEYBOARD_LATCH;
--
--*********************************************************
--
-- keyboard debouncing
--
KEYBOARD_DEBOUNCING : block
        signal D0,D1,EC : STD_LOGIC;
begin
        process (CLK)
        begin
                if CLK'event and CLK='1' then        --CLK rising
                        if EC='1' then                        --CLK enable
                                D1<=D0; D0<=KEY;             --delay
                                FLT <= ((D0 and D1) or FLT) and (D0 or D1); --RS-F/F
                        end if;
                end if;
        end process;
        EC <= PLS3 and not SEQ(1) and not SEQ(0);
end block KEYBOARD_DEBOUNCING;
--
--*********************************************************
--
-- differential signal
--
```

```
KEYBOARD_DIFF : block
        signal D0,D1 : STD_LOGIC;
begin
        process (CLK)
        begin
                if CLK'event and CLK='1' then        --CLK rising
                        D1<=D0; D0<=FLT;                    --delay
                end if;
        end process;
        KIN <= D0 and not D1;                           --differential
end block KEYBOARD_DIFF;
--
--*******************************************************
-- end of architechture
--
end KEYBOARD_ARCH;
```

以下為成果圖



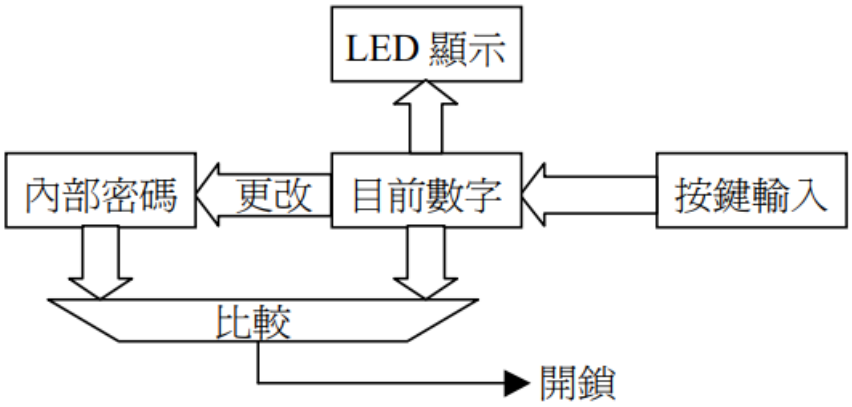


左圖為輸入正確密碼 4321 時的情況，輸入正確密碼時會閃爍，右圖則為輸入錯誤密碼時沒有反應的情況。

四 重點提醒

1. 利用二極體來解決鬼鍵問題。
2. 本次密碼鎖功能主要分三大類
   1. 數字輸入：可以鍵入數字，按錯了可以退位，也可以清除再重來。
   2. 密碼檢查：將顯示數字和內部密碼相比較，相同就開鎖，也可以再上鎖。
   3. 密碼更改：內部的密碼可以清除成"0000"，也可再更改成新密碼。要更改密碼必須先輸入舊密碼開鎖，正確開鎖後才能設定新密碼。

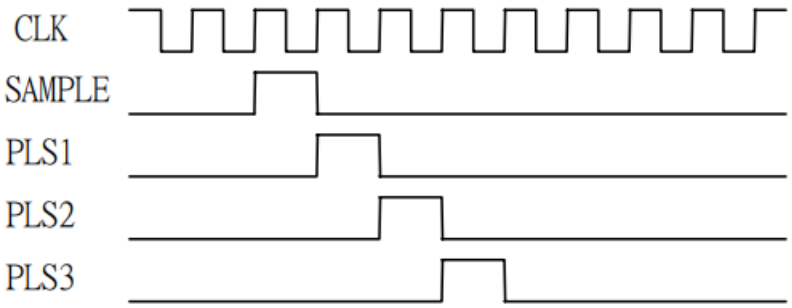
   以下為系統方塊圖:



3. KEYBOARD_SEQUENCE : block --按鍵次序處理電路宣告

```
begin
  process (CLK)                                    --順序邏輯開始
  begin
        if CLK'event and CLK='1' then              --CLK 上緣觸發
              PLS3<=PLS2; PLS2<=PLS1; PLS1<=SAMPLE;  --延遲電路
        end if;
  end process;                                     --順序邏輯結束
end block KEYBOARD_SEQUENCE;
```

此為標準延遲電路其時序圖如下



- 利用 PLS1 來觸發每排的按鍵解碼動作。
- 利用 PLS2 來觸發按鍵暫存電路。
- 利用 PLS3 來做按鍵解碼電路的復歸動作。
- 所以每當 SEQ=00 時，我們就依次執行按鍵解碼、按鍵儲存和解碼復歸的三個 動作。三個動作雖然是依序執行，但是其間只相差一個系統時鐘(4MHz 或是 250ns)，所以幾乎是一瞬間就連續完成三個動作。

五 心得結論

最近的專題內容都是用以前練習過的專題來進行延伸，從入門一路越學越深，活用的深度也是越來越廣，這次除了輸入密碼外，多設計很多的功能，每一項功能都在考驗之前學習的基礎夠不夠扎實，從已知的程式去延伸出更多的寫法，並且去呈現出越來越得多的功能，也讓這堂課變得越來越有趣，不過現在的程式碼是越來越長，考驗自己細心和耐心程度就很重要，現階段要讀懂全部程式碼就是一件不容易的事，不夠細心和耐心可能就會常常找不到錯誤在哪裡。